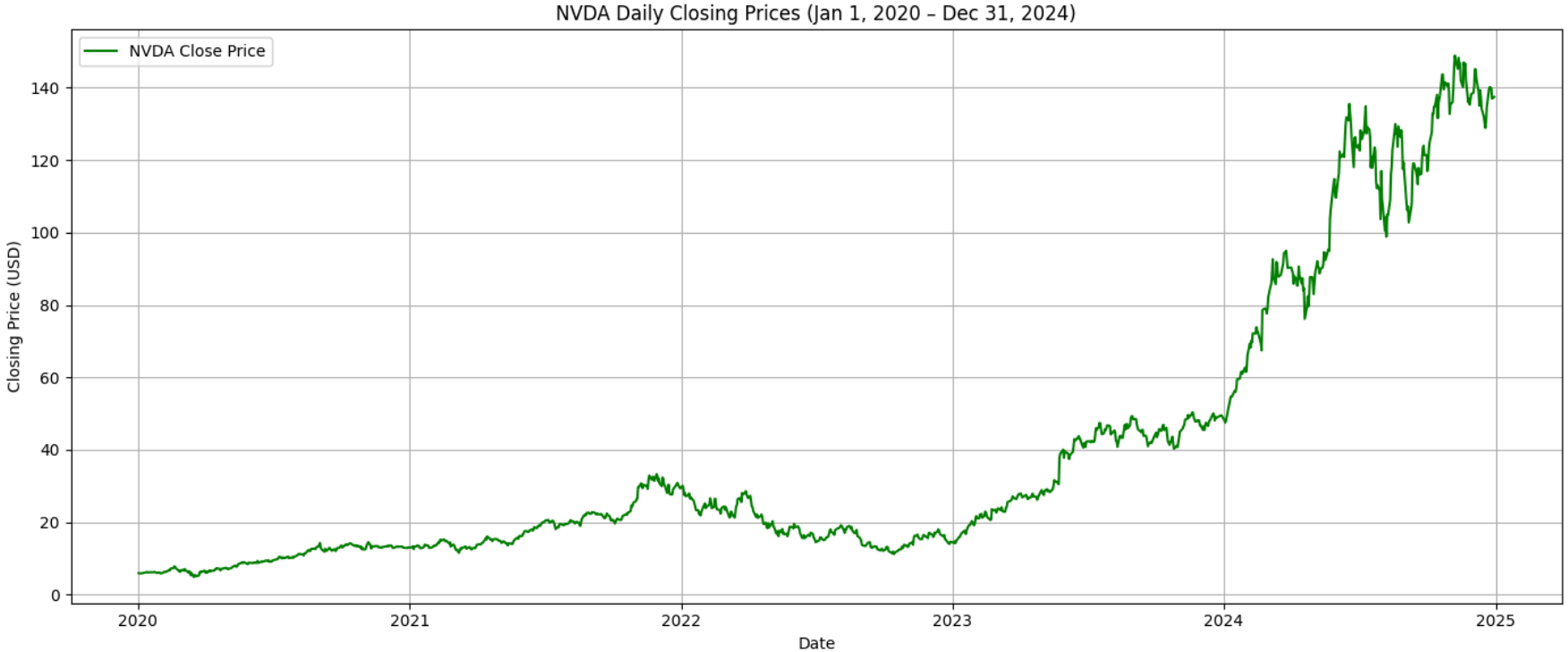
Step 1

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt

# ---------------------------------------------------------------------------
# 1) Fetch NVDA daily historical data from Jan 1, 2020 to Dec 31, 2024
# ---------------------------------------------------------------------------
ticker = "NVDA"
start_date = "2020-01-01"
end_date = "2024-12-31"

# Download data
data = yf.download(ticker, start=start_date, end=end_date)

# ---------------------------------------------------------------------------
# 2) Inspect the dataset (first & last few rows)
# ---------------------------------------------------------------------------
print("=== First 5 rows of NVDA data ===")
print(data.head())
print("\n=== Last 5 rows of NVDA data ===")
print(data.tail())

# ---------------------------------------------------------------------------
# 3) Plot daily closing price over time
# ---------------------------------------------------------------------------
plt.figure(figsize=(14, 6))
plt.plot(data.index, data['Close'], color='green', label='NVDA Close Price')
plt.title('NVDA Daily Closing Prices (Jan 1, 2020 – Dec 31, 2024)')
plt.xlabel('Date')
plt.ylabel('Closing Price (USD)')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# ---------------------------------------------------------------------------
# 4) Optional: Save full dataset to CSV
# ---------------------------------------------------------------------------
output_filename = "NVDA_2020_2024.csv"
data.to_csv(output_filename)
print(f"Saved data to '{output_filename}'")
```

```
/tmp/ipython-input-640912082.py:13: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download(ticker, start=start_date, end=end_date)
[*********************100%***********************]  1 of 1 completed
=== First 5 rows of NVDA data ===
Price          Close      High       Low      Open     Volume
Ticker          NVDA      NVDA      NVDA      NVDA       NVDA
Date
2020-01-02  5.971077  5.971077  5.891683  5.942207  237536000
2020-01-03  5.875505  5.919309  5.826474  5.851363  205384000
2020-01-06  5.900144  5.905370  5.756038  5.782171  262636000
2020-01-07  5.971575  6.017370  5.883468  5.928517  314856000
2020-01-08  5.982774  6.024089  5.927272  5.967343  277108000

=== Last 5 rows of NVDA data ===
Price            Close        High         Low        Open     Volume
Ticker            NVDA        NVDA        NVDA        NVDA       NVDA
Date
2024-12-23  139.631790  139.751752  135.083032  136.242718  176053500
2024-12-24  140.181656  141.861189  138.612078  139.961715  105157000
2024-12-26  139.891724  140.811485  137.692328  139.661791  116205600
2024-12-27  136.972534  138.981994  134.673175  138.512122  170582600
2024-12-30  137.452393  140.231631  133.983341  134.793117  167734700
```




```
Saved data to 'NVDA_2020_2024.csv'
```

```
import yfinance as yf
import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
```

```
data = yf.download("NVDA", start="2020-01-01", end="2024-12-31")

# Log returns
data["log_return"] = np.log(data["Close"] / data["Close"].shift(1))
data.dropna(inplace=True)
```

```
/tmp/ipython-input-235632123.py:1: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download("NVDA", start="2020-01-01", end="2024-12-31")
[*********************100%***********************]  1 of 1 completed
```

```
k = 5  # window size

data["leaky_label"] = (
    data["log_return"]
    .rolling(window=2*k+1, center=True)
    .mean()
)

data.dropna(inplace=True)
```

```
data["ret_lag1"] = data["log_return"].shift(1)
data["ret_lag2"] = data["log_return"].shift(2)
data["ret_lag3"] = data["log_return"].shift(3)

data.dropna(inplace=True)

X = data[["ret_lag1", "ret_lag2", "ret_lag3"]]
y = data["leaky_label"]
```

```
split_date = "2023-01-01"

X_train = X.loc[:split_date]
X_test  = X.loc[split_date:]

y_train = y.loc[:split_date]
```

```
y_test  = y.loc[split_date:]
```

```
model = RandomForestRegressor(
    n_estimators=300,
    max_depth=6,
    random_state=42
)

model.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(max_depth=6, n_estimators=300, random_state=42)
```

```
train_pred = model.predict(X_train)
test_pred  = model.predict(X_test)

print("Train R²:", r2_score(y_train, train_pred))
print("Test  R²:", r2_score(y_test, test_pred))
```

```
Train R²: 0.5309280099934965
Test  R²: 0.11391720311702502
```

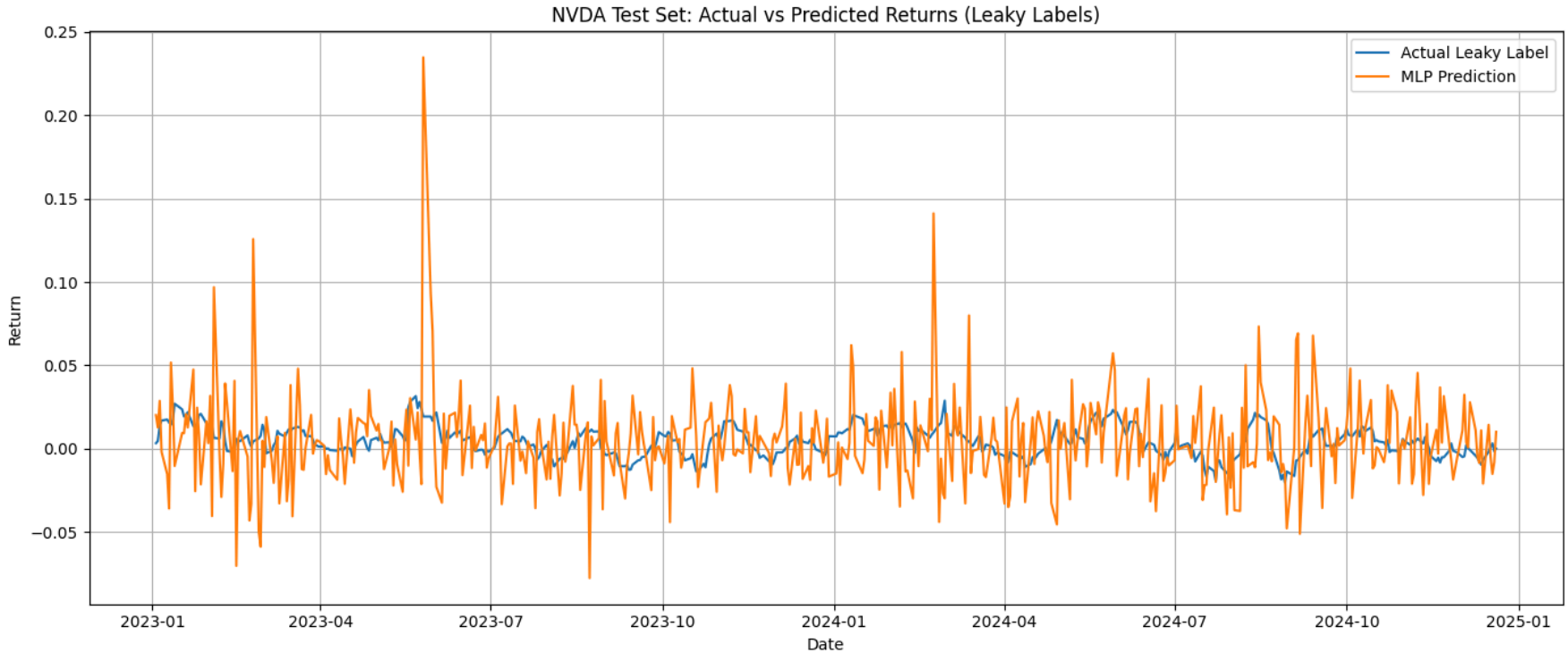
```
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error

# Select lagged return features (causal-looking but labels are not)
X = data[[
    "ret_lag1",
    "ret_lag2",
    "ret_lag3"
]]

# Leaky target variable
y = data["leaky_label"]

# Single time-based split
split_date = "2023-01-01"

X_train = X.loc[:split_date]
X_test  = X.loc[split_date:]

y_train = y.loc[:split_date]
```

```
y_test  = y.loc[split_date:]

mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPRegressor(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        solver="adam",
        alpha=1e-4,
        max_iter=2000,
        random_state=42
    ))
])

mlp_pipeline.fit(X_train, y_train)

# Predictions
train_pred = mlp_pipeline.predict(X_train)
test_pred  = mlp_pipeline.predict(X_test)

# Performance metrics
train_r2 = r2_score(y_train, train_pred)
test_r2  = r2_score(y_test, test_pred)

train_mse = mean_squared_error(y_train, train_pred)
test_mse  = mean_squared_error(y_test, test_pred)

print("MLP Train R²:", train_r2)
print("MLP Test  R²:", test_r2)
print("MLP Train MSE:", train_mse)
print("MLP Test  MSE:", test_mse)
```

```
MLP Train R²: -6.125267426761353
MLP Test  R²: -8.0574462852766
MLP Train MSE: 0.0007004983421367735
MLP Test  MSE: 0.0007186821431314641
```

```
import matplotlib.pyplot as plt

# Convert predictions to pandas Series for alignment
y_test_pred = pd.Series(
    test_pred,
    index=y_test.index
)

# Plot actual vs predicted leaky labels
plt.figure(figsize=(14, 6))
plt.plot(y_test.index, y_test, label="Actual Leaky Label")
```

```
plt.plot(y_test_pred.index, y_test_pred, label="MLP Prediction")
plt.title("NVDA Test Set: Actual vs Predicted Returns (Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Return")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import r2_score, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

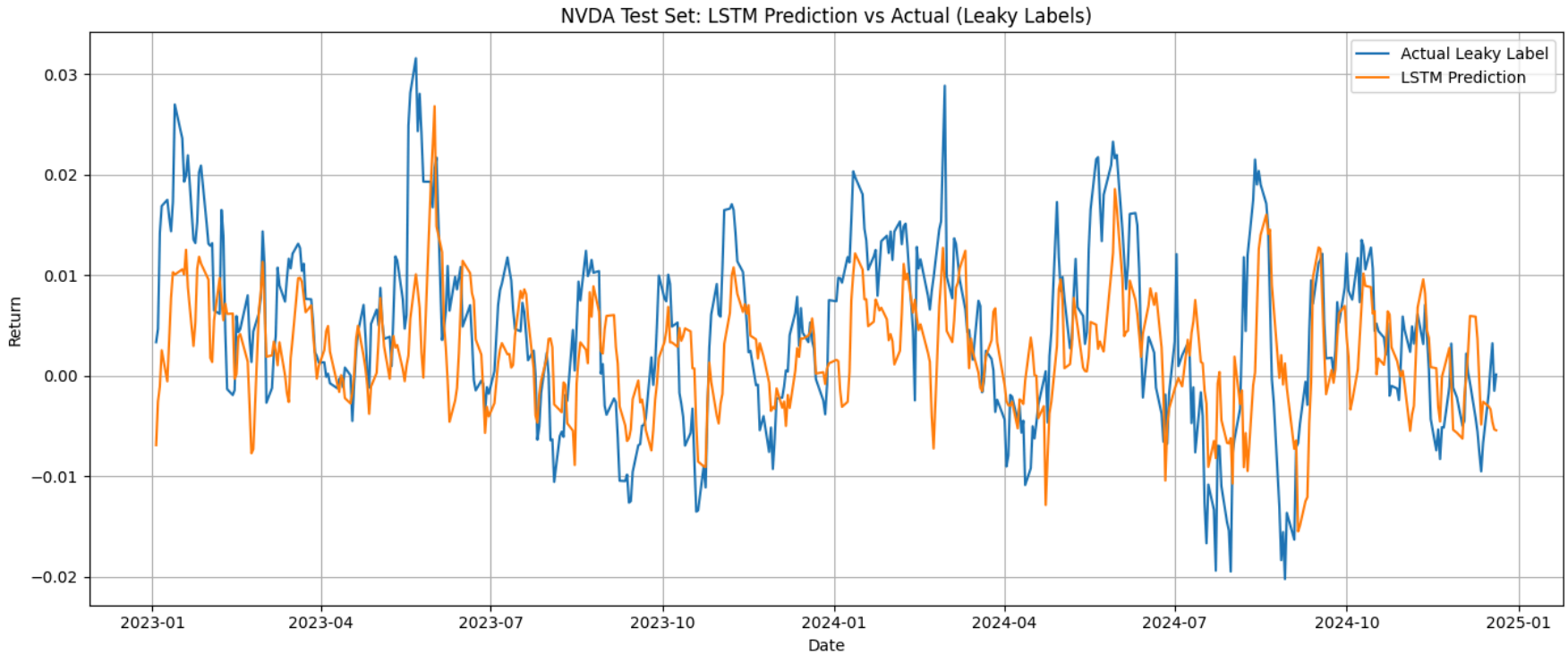
```
from tensorflow.keras.optimizers import Adam

# ----------------------------
# Prepare sequential data
# ----------------------------
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]
target_col = "leaky_label"

X_raw = data[feature_cols].values
y_raw = data[target_col].values
dates = data.index.values

seq_len = 10

X_seq, y_seq, date_seq = [], [], []

for i in range(seq_len, len(X_raw)):
    X_seq.append(X_raw[i - seq_len:i])
    y_seq.append(y_raw[i])
    date_seq.append(dates[i])

X_seq = np.array(X_seq)
y_seq = np.array(y_seq)
date_seq = np.array(date_seq)

# ----------------------------
# Single time-based train/test split
# ----------------------------
split_date = np.datetime64("2023-01-01")

train_mask = date_seq < split_date
test_mask  = date_seq >= split_date

X_train, X_test = X_seq[train_mask], X_seq[test_mask]
y_train, y_test = y_seq[train_mask], y_seq[test_mask]
dates_train, dates_test = date_seq[train_mask], date_seq[test_mask]

# ----------------------------
# Build LSTM model
# ----------------------------
model = Sequential([
    LSTM(64, input_shape=(seq_len, X_train.shape[2])),
    Dense(1)
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse"
)
```

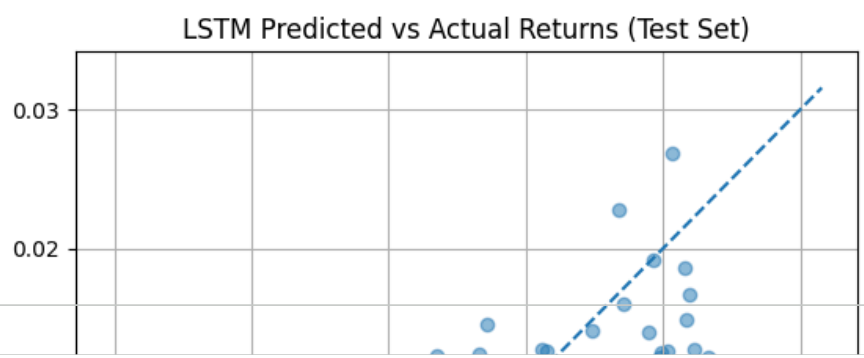
```
# ----------------------------
# Train model
# ----------------------------
model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    shuffle=False,
    verbose=1
)

# ----------------------------
# Predictions
# ----------------------------
train_pred = model.predict(X_train).flatten()
test_pred  = model.predict(X_test).flatten()

# ----------------------------
# Metrics: R² and MSE
# ----------------------------
train_r2 = r2_score(y_train, train_pred)
test_r2  = r2_score(y_test, test_pred)

train_mse = mean_squared_error(y_train, train_pred)
test_mse  = mean_squared_error(y_test, test_pred)

print("Train R²:", train_r2)
print("Test  R²:", test_r2)
print("Train MSE:", train_mse)
print("Test  MSE:", test_mse)

# ----------------------------
# Plot 1: Time series (Test set)
# ----------------------------
plt.figure(figsize=(14, 6))
plt.plot(dates_test, y_test, label="Actual Leaky Label")
plt.plot(dates_test, test_pred, label="LSTM Prediction")
plt.title("NVDA Test Set: LSTM Prediction vs Actual (Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Return")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ----------------------------
```

```
# Plot 2: Predicted vs Actual scatter (Test set)
# ----------------------------
plt.figure(figsize=(6, 6))
plt.scatter(y_test, test_pred, alpha=0.5)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)
plt.title("LSTM Predicted vs Actual Returns (Test Set)")
plt.xlabel("Actual Leaky Label")
plt.ylabel("Predicted Return")
plt.grid(True)
plt.tight_layout()
plt.show()

# ----------------------------
# Plot 3: Train vs Test comparison
# ----------------------------
plt.figure(figsize=(14, 6))
plt.plot(dates_train, y_train, label="Train Actual", alpha=0.4)
plt.plot(dates_train, train_pred, label="Train Predicted", alpha=0.4)
plt.plot(dates_test, y_test, label="Test Actual", linewidth=2)
plt.plot(dates_test, test_pred, label="Test Predicted", linewidth=2)
plt.title("LSTM Predictions on Train and Test Sets (Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Return")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer
  super().__init__(**kwargs)
Epoch 1/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 1.7827e-04 - val_loss: 8.3860e-05
Epoch 2/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 9.0021e-05 - val_loss: 9.8434e-05
Epoch 3/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.0343e-04 - val_loss: 1.0151e-04
Epoch 4/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 1.0754e-04 - val_loss: 8.7565e-05
Epoch 5/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.0211e-04 - val_loss: 7.5444e-05
Epoch 6/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 9.3245e-05 - val_loss: 7.0537e-05
Epoch 7/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 8.6316e-05 - val_loss: 6.8846e-05
Epoch 8/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 8.2240e-05 - val_loss: 6.8645e-05
Epoch 9/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 8.0703e-05 - val_loss: 6.8737e-05
Epoch 10/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 8.0366e-05 - val_loss: 6.8700e-05
Epoch 11/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 8.0107e-05 - val_loss: 6.8650e-05
Epoch 12/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.9799e-05 - val_loss: 6.8590e-05
Epoch 13/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.9456e-05 - val_loss: 6.8554e-05
Epoch 14/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.9143e-05 - val_loss: 6.8517e-05
Epoch 15/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.8840e-05 - val_loss: 6.8494e-05
Epoch 16/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.8563e-05 - val_loss: 6.8469e-05
Epoch 17/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.8295e-05 - val_loss: 6.8457e-05
Epoch 18/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.8053e-05 - val_loss: 6.8442e-05
Epoch 19/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.7819e-05 - val_loss: 6.8440e-05
Epoch 20/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.7615e-05 - val_loss: 6.8431e-05
Epoch 21/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.7416e-05 - val_loss: 6.8440e-05
Epoch 22/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.7258e-05 - val_loss: 6.8435e-05
Epoch 23/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.7100e-05 - val_loss: 6.8453e-05
Epoch 24/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.6991e-05 - val_loss: 6.8442e-05
Epoch 25/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.6848e-05 - val_loss: 6.8455e-05
Epoch 26/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.6744e-05 - val_loss: 6.8439e-05
```

```
Epoch 27/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.6597e-05 - val_loss: 6.8431e-05
Epoch 28/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 7.6452e-05 - val_loss: 6.8388e-05
Epoch 29/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.6203e-05 - val_loss: 6.8365e-05
Epoch 30/30
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.5972e-05 - val_loss: 6.8338e-05
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
Train R²: 0.3604301079181387
Test  R²: 0.29171330737398493
Train MSE: 6.365724102345829e-05
Test  MSE: 5.6200498702975865e-05
```





```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam

# ------------------------------------------------
# Prepare raw features and labels
# ------------------------------------------------
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]
target_col = "leaky_label"

X_raw = data[feature_cols].values
y_raw = data[target_col].values
dates = data.index.values

# ------------------------------------------------
# Scale features to [-1, 1] for GAF
# ------------------------------------------------
scaler = MinMaxScaler(feature_range=(-1, 1))
X_scaled = scaler.fit_transform(X_raw)

# ------------------------------------------------
# Gramian Angular Field function
# ------------------------------------------------
def gramian_angular_field(ts):
    ts = np.clip(ts, -1, 1)
    phi = np.arccos(ts)
    return np.cos(phi[:, None] + phi[None, :])

# ------------------------------------------------
# Build multi-channel GAF images
# ------------------------------------------------
window_size = 16

X_gaf = []
y_seq = []
date_seq = []

for i in range(window_size, len(X_scaled)):
    gaf_channels = []
    for j in range(X_scaled.shape[1]):
        ts_window = X_scaled[i - window_size:i, j]
        gaf_channels.append(gramian_angular_field(ts_window))
    X_gaf.append(np.stack(gaf_channels, axis=-1))
```

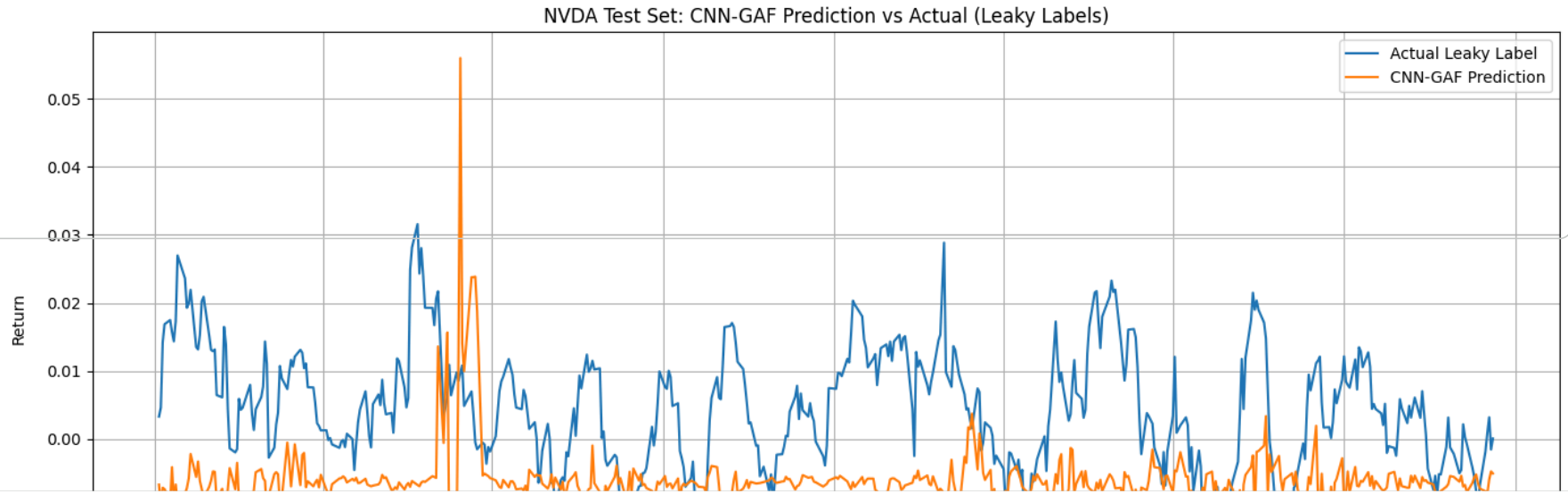
```
    y_seq.append(y_raw[i])
    date_seq.append(dates[i])

X_gaf = np.array(X_gaf)
y_seq = np.array(y_seq)
date_seq = np.array(date_seq)

# ------------------------------------------------
# Single time-based train/test split
# ------------------------------------------------
split_date = np.datetime64("2023-01-01")

train_mask = date_seq < split_date
test_mask  = date_seq >= split_date

X_train, X_test = X_gaf[train_mask], X_gaf[test_mask]
y_train, y_test = y_seq[train_mask], y_seq[test_mask]
dates_train, dates_test = date_seq[train_mask], date_seq[test_mask]

# ------------------------------------------------
# Build CNN model
# ------------------------------------------------
model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=X_train.shape[1:]),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(1)
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse"
)

# ------------------------------------------------
# Train model
# ------------------------------------------------
model.fit(
    X_train,
    y_train,
    epochs=40,
    batch_size=32,
    validation_split=0.1,
    shuffle=False,
    verbose=1
)
```

```
# ------------------------------------------------
# Predictions
# ------------------------------------------------
train_pred = model.predict(X_train).flatten()
test_pred  = model.predict(X_test).flatten()

# ------------------------------------------------
# Metrics
# ------------------------------------------------
train_r2 = r2_score(y_train, train_pred)
test_r2  = r2_score(y_test, test_pred)
train_mse = mean_squared_error(y_train, train_pred)
test_mse  = mean_squared_error(y_test, test_pred)

print("Train R²:", train_r2)
print("Test  R²:", test_r2)
print("Train MSE:", train_mse)
print("Test  MSE:", test_mse)

# ------------------------------------------------
# Plot 1: Time series (Test set)
# ------------------------------------------------
plt.figure(figsize=(14, 6))
plt.plot(dates_test, y_test, label="Actual Leaky Label")
plt.plot(dates_test, test_pred, label="CNN-GAF Prediction")
plt.title("NVDA Test Set: CNN-GAF Prediction vs Actual (Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Return")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# Plot 2: Predicted vs Actual scatter
# ------------------------------------------------
plt.figure(figsize=(6, 6))
plt.scatter(y_test, test_pred, alpha=0.5)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)
plt.title("CNN-GAF Predicted vs Actual Returns (Test Set)")
plt.xlabel("Actual Leaky Label")
plt.ylabel("Predicted Return")
plt.grid(True)
plt.tight_layout()
```

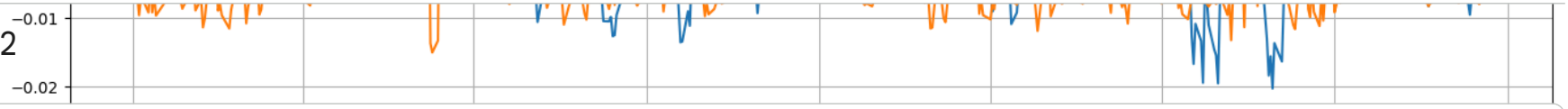
```
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - loss: 0.0117 - val_loss: 6.6776e-04
Epoch 2/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0047 - val_loss: 2.7555e-04
Epoch 3/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 5.2051e-04 - val_loss: 5.0827e-04
Epoch 4/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 3.5566e-04 - val_loss: 1.2407e-04
Epoch 5/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.5200e-04 - val_loss: 1.1759e-04
Epoch 6/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.1822e-04 - val_loss: 1.1701e-04
Epoch 7/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.1849e-04 - val_loss: 1.1710e-04
Epoch 8/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 1.1411e-04 - val_loss: 1.1738e-04
Epoch 9/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 1.1535e-04 - val_loss: 1.1741e-04
Epoch 10/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 1.1344e-04 - val_loss: 1.1673e-04
Epoch 11/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 1.1107e-04 - val_loss: 1.1702e-04
Epoch 12/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.0964e-04 - val_loss: 1.1792e-04
Epoch 13/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.0996e-04 - val_loss: 1.1961e-04
Epoch 14/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.1199e-04 - val_loss: 1.2268e-04
Epoch 15/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.1438e-04 - val_loss: 1.3096e-04
Epoch 16/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2230e-04 - val_loss: 1.4917e-04
Epoch 17/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3402e-04 - val_loss: 2.0127e-04
Epoch 18/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.6239e-04 - val_loss: 3.4536e-04
Epoch 19/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 2.1746e-04 - val_loss: 6.2501e-04
Epoch 20/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 3.0760e-04 - val_loss: 4.3709e-04
Epoch 21/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 2.9583e-04 - val_loss: 1.5173e-04
Epoch 22/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.5962e-04 - val_loss: 1.8992e-04
Epoch 23/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.4551e-04 - val_loss: 1.9482e-04
Epoch 24/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.6748e-04 - val_loss: 1.7611e-04
Epoch 25/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3040e-04 - val_loss: 1.9124e-04
Epoch 26/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.4538e-04 - val_loss: 1.7803e-04
```

```
Epoch 27/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2964e-04 - val_loss: 1.8611e-04
Epoch 28/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3490e-04 - val_loss: 1.7844e-04
Epoch 29/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2925e-04 - val_loss: 1.8331e-04
Epoch 30/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 1.3017e-04 - val_loss: 1.7837e-04
Epoch 31/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2826e-04 - val_loss: 1.8067e-04
Epoch 32/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.2516e-04 - val_loss: 1.7894e-04
Epoch 33/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2849e-04 - val_loss: 1.8483e-04
Epoch 34/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2494e-04 - val_loss: 1.7808e-04
Epoch 35/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3226e-04 - val_loss: 1.8755e-04
Epoch 36/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.2573e-04 - val_loss: 1.7914e-04
Epoch 37/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3879e-04 - val_loss: 1.9484e-04
Epoch 38/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.2746e-04 - val_loss: 1.8115e-04
Epoch 39/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.3961e-04 - val_loss: 2.0351e-04
Epoch 40/40
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 1.3186e-04 - val_loss: 1.7666e-04
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
Train R²: -0.5224775953938348
Test  R²: -1.6087851788965768
Train MSE: 0.00015235171691330209
Test  MSE: 0.0002069995520025031
```



Step 2



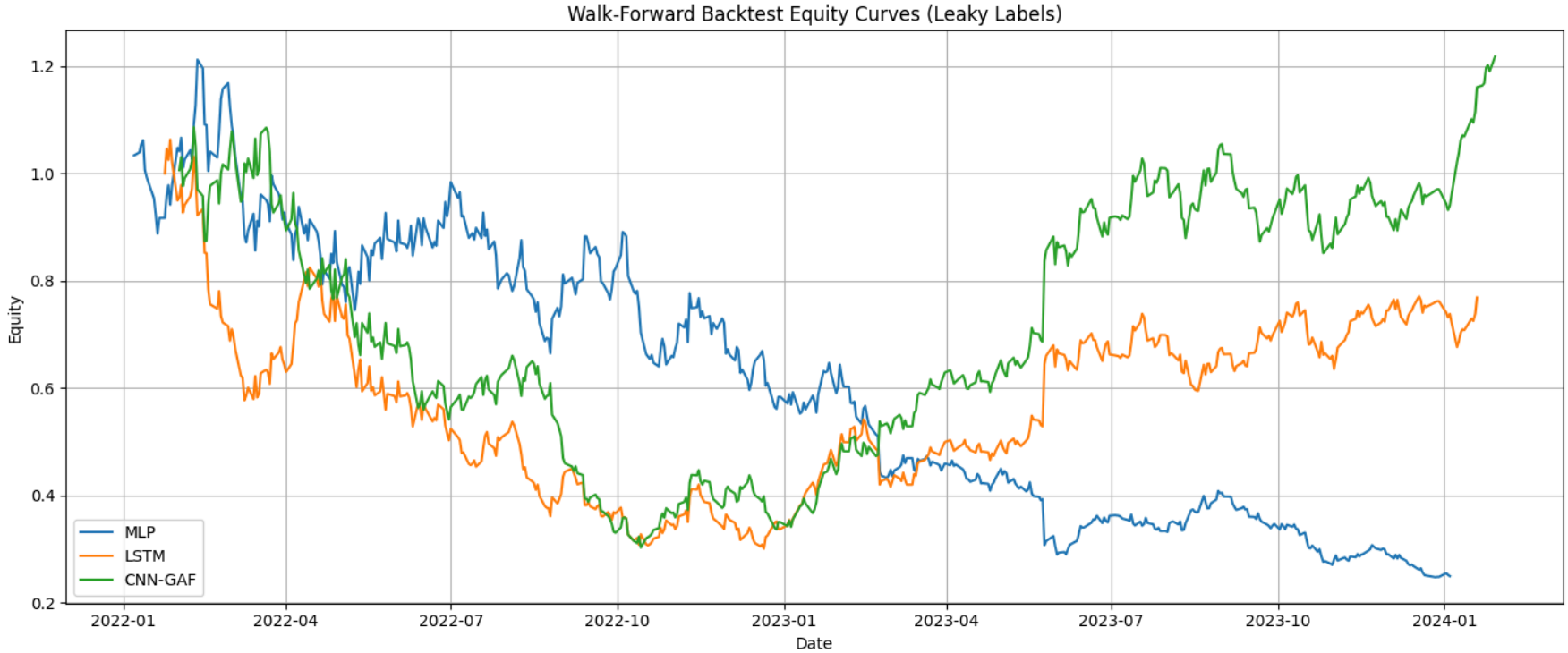

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPRegressor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.optimizers import Adam

# ========================================================
# 0. Common settings
# ========================================================
train_size = 500
test_size = 500
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]

returns = data["log_return"]

# ========================================================
# 1. Build feature matrices
# ========================================================

# -------- MLP (2D) --------
X_mlp = data[feature_cols].values
y_mlp = data["leaky_label"].values
returns_mlp = returns

# -------- LSTM (3D sequence) --------
seq_len = 10

X_lstm, y_lstm, returns_lstm = [], [], []

X_raw = X_mlp
y_raw = y_mlp

for i in range(seq_len, len(X_raw)):
    X_lstm.append(X_raw[i - seq_len:i])
    y_lstm.append(y_raw[i])
    returns_lstm.append(returns.iloc[i])

X_lstm = np.array(X_lstm)
```

```
y_lstm = np.array(y_lstm)
returns_lstm = pd.Series(returns_lstm, index=data.index[seq_len:])

# -------- CNN-GAF (4D image) --------
def gramian_angular_field(ts):
    ts = np.clip(ts, -1, 1)
    phi = np.arccos(ts)
    return np.cos(phi[:, None] + phi[None, :])

scaler = MinMaxScaler(feature_range=(-1, 1))
X_scaled = scaler.fit_transform(X_raw)

window_size = 16
X_gaf, y_gaf, returns_gaf = [], [], []

for i in range(window_size, len(X_scaled)):
    channels = []
    for j in range(X_scaled.shape[1]):
        ts_window = X_scaled[i - window_size:i, j]
        channels.append(gramian_angular_field(ts_window))
    X_gaf.append(np.stack(channels, axis=-1))
    y_gaf.append(y_raw[i])
    returns_gaf.append(returns.iloc[i])

X_gaf = np.array(X_gaf)
y_gaf = np.array(y_gaf)
returns_gaf = pd.Series(returns_gaf, index=data.index[window_size:])

# ========================================================
# 2. Model builders
# ========================================================
def build_mlp():
    return Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(hidden_layer_sizes=(64, 32),
                             max_iter=1000,
                             random_state=42))
    ])

def build_lstm(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

def build_cnn(input_shape):
    model = Sequential([
```

```
        Conv2D(32, (3,3), activation="relu", input_shape=input_shape),
        MaxPooling2D((2,2)),
        Conv2D(64, (3,3), activation="relu"),
        MaxPooling2D((2,2)),
        Flatten(),
        Dense(64, activation="relu"),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

# ==========================================================
# 3. Walk-forward backtest engine (non-anchored)
# ==========================================================
def walk_forward_backtest(X, y, returns, model_type):
    strategy_returns = []
    idx = returns.index
    start = 0

    while start + train_size + test_size <= len(X):
        tr_slice = slice(start, start + train_size)
        te_slice = slice(start + train_size, start + train_size + test_size)

        X_train, X_test = X[tr_slice], X[te_slice]
        y_train, y_test = y[tr_slice], y[te_slice]
        ret_test = returns.iloc[te_slice]

        if model_type == "mlp":
            model = build_mlp()
            model.fit(X_train, y_train)
            preds = model.predict(X_test)

        elif model_type == "lstm":
            model = build_lstm((X.shape[1], X.shape[2]))
            model.fit(X_train, y_train,
                      epochs=20, batch_size=32,
                      shuffle=False, verbose=0)
            preds = model.predict(X_test).flatten()

        elif model_type == "cnn":
            model = build_cnn(X.shape[1:])
            model.fit(X_train, y_train,
                      epochs=20, batch_size=32,
                      shuffle=False, verbose=0)
            preds = model.predict(X_test).flatten()

        signals = np.sign(preds)
        strat_ret = signals * ret_test.values
```

```
        strategy_returns.extend(strat_ret)
        start += test_size

    return pd.Series(strategy_returns, index=idx[train_size:train_size+len(strategy_returns)])

# ========================================================
# 4. Run backtests
# ========================================================
mlp_returns  = walk_forward_backtest(X_mlp,  y_mlp,  returns_mlp,  "mlp")
lstm_returns = walk_forward_backtest(X_lstm, y_lstm, returns_lstm, "lstm")
cnn_returns  = walk_forward_backtest(X_gaf,  y_gaf,  returns_gaf,  "cnn")

# ========================================================
# 5. Performance summary
# ========================================================
def summarize(returns):
    equity = (1 + returns).cumprod()
    total_return = equity.iloc[-1] - 1
    sharpe = np.sqrt(252) * returns.mean() / returns.std()
    max_dd = (equity / equity.cummax() - 1).min()
    return total_return, sharpe, max_dd

summary = pd.DataFrame({
    "MLP": summarize(mlp_returns),
    "LSTM": summarize(lstm_returns),
    "CNN-GAF": summarize(cnn_returns)
}, index=["Total Return", "Sharpe Ratio", "Max Drawdown"])

print(summary)

# ========================================================
# 6. Plot equity curves
# ========================================================
plt.figure(figsize=(14,6))
plt.plot((1+mlp_returns).cumprod(), label="MLP")
plt.plot((1+lstm_returns).cumprod(), label="LSTM")
plt.plot((1+cnn_returns).cumprod(), label="CNN-GAF")
plt.title("Walk-Forward Backtest Equity Curves (Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Equity")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer
  super().__init__(**kwargs)
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
                  MLP      LSTM   CNN-GAF
Total Return -0.750232 -0.231142  0.217991
Sharpe Ratio -0.973701  0.038300  0.453278
Max Drawdown -0.795552 -0.717062 -0.721082
```




```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.pipeline import Pipeline
```

```
from sklearn.neural_network import MLPRegressor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.optimizers import Adam

# ========================================================
# 0. Common settings
# ========================================================
train_size = 500
test_size = 100
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]

returns = data["log_return"]

# ========================================================
# 1. Build feature matrices
# ========================================================

# -------- MLP --------
X_mlp = data[feature_cols].values
y_mlp = data["leaky_label"].values
returns_mlp = returns

# -------- LSTM --------
seq_len = 10
X_lstm, y_lstm, returns_lstm = [], [], []

for i in range(seq_len, len(X_mlp)):
    X_lstm.append(X_mlp[i - seq_len:i])
    y_lstm.append(y_mlp[i])
    returns_lstm.append(returns.iloc[i])

X_lstm = np.array(X_lstm)
y_lstm = np.array(y_lstm)
returns_lstm = pd.Series(returns_lstm, index=data.index[seq_len:])

# -------- CNN-GAF --------
def gramian_angular_field(ts):
    ts = np.clip(ts, -1, 1)
    phi = np.arccos(ts)
    return np.cos(phi[:, None] + phi[None, :])

scaler = MinMaxScaler(feature_range=(-1, 1))
X_scaled = scaler.fit_transform(X_mlp)

window_size = 16
X_gaf, y_gaf, returns_gaf = [], [], []
```

```
for i in range(window_size, len(X_scaled)):
    channels = []
    for j in range(X_scaled.shape[1]):
        ts_window = X_scaled[i - window_size:i, j]
        channels.append(gramian_angular_field(ts_window))
    X_gaf.append(np.stack(channels, axis=-1))
    y_gaf.append(y_mlp[i])
    returns_gaf.append(returns.iloc[i])

X_gaf = np.array(X_gaf)
y_gaf = np.array(y_gaf)
returns_gaf = pd.Series(returns_gaf, index=data.index[window_size:])

# ========================================================
# 2. Model builders
# ========================================================
def build_mlp():
    return Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(hidden_layer_sizes=(64, 32),
                             max_iter=1000,
                             random_state=42))
    ])

def build_lstm(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

def build_cnn(input_shape):
    model = Sequential([
        Conv2D(32, (3,3), activation="relu", input_shape=input_shape),
        MaxPooling2D((2,2)),
        Conv2D(64, (3,3), activation="relu"),
        MaxPooling2D((2,2)),
        Flatten(),
        Dense(64, activation="relu"),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

# ========================================================
# 3. Walk-forward backtest engine (non-anchored)
# ========================================================
def walk_forward_backtest(X, y, returns, model_type):
```

```
    strategy_returns = []
    index_out = []
    start = 0

    while start + train_size + test_size <= len(X):

        tr = slice(start, start + train_size)
        te = slice(start + train_size, start + train_size + test_size)

        X_train, X_test = X[tr], X[te]
        y_train = y[tr]
        ret_test = returns.iloc[te]

        if model_type == "mlp":
            model = build_mlp()
            model.fit(X_train, y_train)
            preds = model.predict(X_test)

        elif model_type == "lstm":
            model = build_lstm((X.shape[1], X.shape[2]))
            model.fit(X_train, y_train,
                      epochs=20, batch_size=32,
                      shuffle=False, verbose=0)
            preds = model.predict(X_test).flatten()

        elif model_type == "cnn":
            model = build_cnn(X.shape[1:])
            model.fit(X_train, y_train,
                      epochs=20, batch_size=32,
                      shuffle=False, verbose=0)
            preds = model.predict(X_test).flatten()

        signals = np.sign(preds)
        strat_ret = signals * ret_test.values

        strategy_returns.extend(strat_ret)
        index_out.extend(ret_test.index)

        start += test_size

    return pd.Series(strategy_returns, index=index_out)

# ========================================================
# 4. Run backtests
# ========================================================
mlp_returns  = walk_forward_backtest(X_mlp,  y_mlp,  returns_mlp,  "mlp")
lstm_returns = walk_forward_backtest(X_lstm, y_lstm, returns_lstm, "lstm")
cnn_returns  = walk_forward_backtest(X_gaf,  y_gaf,  returns_gaf,  "cnn")
```

```
# ========================================================
# 5. Performance summary
# ========================================================
def summarize(returns):
    equity = (1 + returns).cumprod()
    total_return = equity.iloc[-1] - 1
    sharpe = np.sqrt(252) * returns.mean() / returns.std()
    max_dd = (equity / equity.cummax() - 1).min()
    return total_return, sharpe, max_dd

summary = pd.DataFrame({
    "MLP": summarize(mlp_returns),
    "LSTM": summarize(lstm_returns),
    "CNN-GAF": summarize(cnn_returns)
}, index=["Total Return", "Sharpe Ratio", "Max Drawdown"])

print(summary)

# ========================================================
# 6. Plot equity curves
# ========================================================
plt.figure(figsize=(14,6))
plt.plot((1+mlp_returns).cumprod(), label="MLP")
plt.plot((1+lstm_returns).cumprod(), label="LSTM")
plt.plot((1+cnn_returns).cumprod(), label="CNN-GAF")
plt.title("Walk-Forward Backtest (Train=500, Test=100, Leaky Labels)")
plt.xlabel("Date")
plt.ylabel("Equity")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer
  super().__init__(**kwargs)
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
WARNING:tensorflow:5 out of the last 21 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x789e
1/4 ━━━━━━━━━━━━━━━━━━━━ 1s 593ms/stepWARNING:tensorflow:6 out of the last 24 calls to <function TensorFlowTrainer.make_predict_function.<locals>.
4/4 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
```

Step 3

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPRegressor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.optimizers import Adam

# ========================================================
# 0. Causal (clean) label construction
# ========================================================
data["clean_label"] = data["log_return"].shift(-1)
data = data.dropna()

# ========================================================
# 1. Feature matrices
# ========================================================
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]
returns = data["log_return"]

# -------- MLP --------
X_mlp = data[feature_cols].values
y_mlp = data["clean_label"].values
returns_mlp = returns

# -------- LSTM --------
seq_len = 10
X_lstm, y_lstm, returns_lstm = [], [], []

for i in range(seq_len, len(X_mlp)):
    X_lstm.append(X_mlp[i - seq_len:i])
    y_lstm.append(y_mlp[i])
    returns_lstm.append(returns.iloc[i])

X_lstm = np.array(X_lstm)
y_lstm = np.array(y_lstm)
returns_lstm = pd.Series(returns_lstm, index=data.index[seq_len:])
```

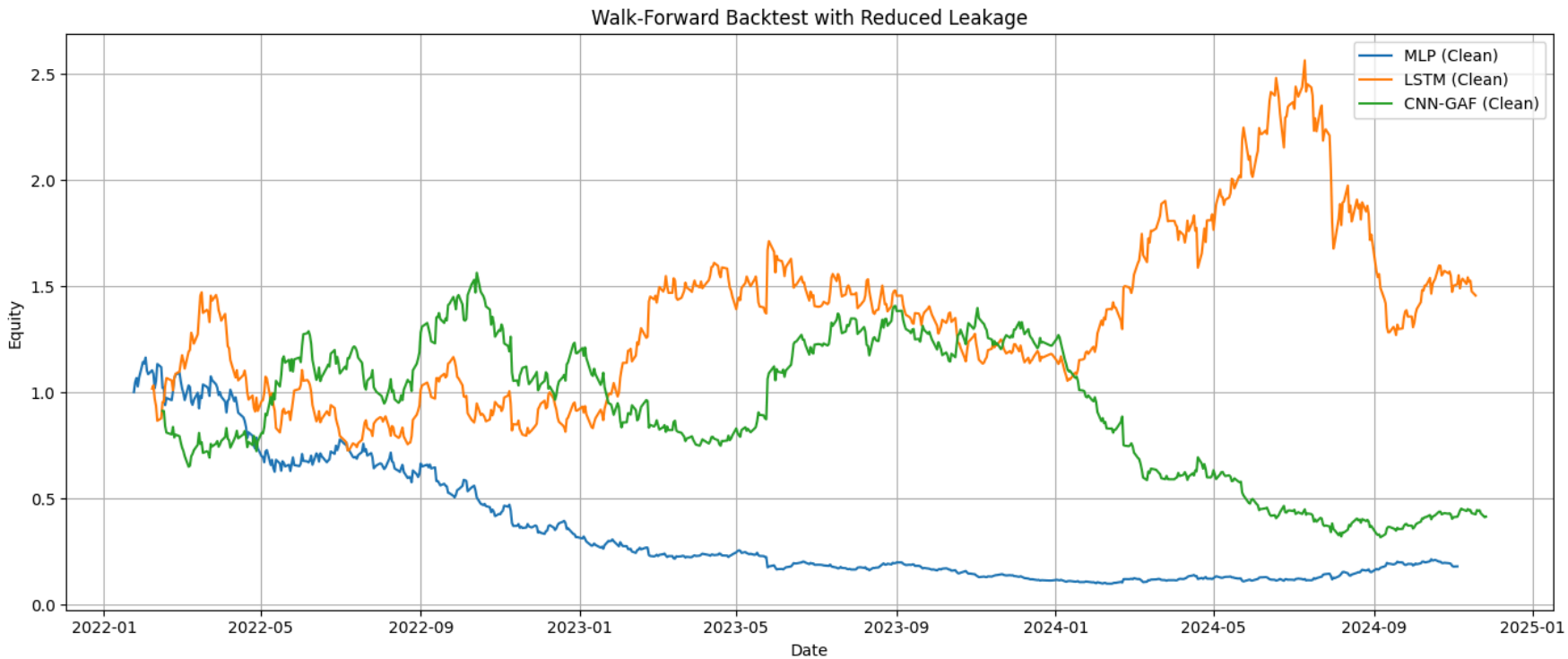
```
# -------- CNN-GAF --------
def gramian_angular_field(ts):
    ts = np.clip(ts, -1, 1)
    phi = np.arccos(ts)
    return np.cos(phi[:, None] + phi[None, :])

scaler = MinMaxScaler(feature_range=(-1, 1))
X_scaled = scaler.fit_transform(X_mlp)

window_size = 16
X_gaf, y_gaf, returns_gaf = [], [], []

for i in range(window_size, len(X_scaled)):
    channels = []
    for j in range(X_scaled.shape[1]):
        ts_window = X_scaled[i - window_size:i, j]
        channels.append(gramian_angular_field(ts_window))
    X_gaf.append(np.stack(channels, axis=-1))
    y_gaf.append(y_mlp[i])
    returns_gaf.append(returns.iloc[i])

X_gaf = np.array(X_gaf)
y_gaf = np.array(y_gaf)
returns_gaf = pd.Series(returns_gaf, index=data.index[window_size:])

# ========================================================
# 2. Model builders
# ========================================================
def build_mlp():
    return Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(64, 32),
            max_iter=1000,
            random_state=42
        ))
    ])

def build_lstm(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

def build_cnn(input_shape):
    model = Sequential([
        Conv2D(32, (3, 3), activation="relu", input_shape=input_shape),
```

```
            MaxPooling2D((2, 2)),
            Conv2D(64, (3, 3), activation="relu"),
            MaxPooling2D((2, 2)),
            Flatten(),
            Dense(64, activation="relu"),
            Dense(1)
        ])
        model.compile(optimizer=Adam(0.001), loss="mse")
        return model

# ========================================================
# 3. Walk-forward backtest with purging and embargo
# ========================================================
def walk_forward_backtest_purged(
    X, y, returns, model_type,
    train_size=500,
    test_size=100,
    purge_size=10,
    embargo_size=10
):
    strategy_returns = []
    index_out = []
    start = 0

    while start + train_size + purge_size + test_size + embargo_size <= len(X):

        train_start = start
        train_end = start + train_size - purge_size
        test_start = start + train_size + purge_size
        test_end = test_start + test_size

        X_train = X[train_start:train_end]
        y_train = y[train_start:train_end]
        X_test = X[test_start:test_end]
        ret_test = returns.iloc[test_start:test_end]

        if model_type == "mlp":
            model = build_mlp()
            model.fit(X_train, y_train)
            preds = model.predict(X_test)

        elif model_type == "lstm":
            model = build_lstm((X.shape[1], X.shape[2]))
            model.fit(
                X_train, y_train,
                epochs=20,
                batch_size=32,
                shuffle=False,
                verbose=0
```

```
            )
            preds = model.predict(X_test).flatten()

        elif model_type == "cnn":
            model = build_cnn(X.shape[1:])
            model.fit(
                X_train, y_train,
                epochs=20,
                batch_size=32,
                shuffle=False,
                verbose=0
            )
            preds = model.predict(X_test).flatten()

        signals = np.sign(preds)
        strat_ret = signals * ret_test.values

        strategy_returns.extend(strat_ret)
        index_out.extend(ret_test.index)

        start += test_size

    return pd.Series(strategy_returns, index=index_out)

# ========================================================
# 4. Run leakage-reduced backtests
# ========================================================
mlp_clean = walk_forward_backtest_purged(X_mlp, y_mlp, returns_mlp, "mlp")
lstm_clean = walk_forward_backtest_purged(X_lstm, y_lstm, returns_lstm, "lstm")
cnn_clean = walk_forward_backtest_purged(X_gaf, y_gaf, returns_gaf, "cnn")

# ========================================================
# 5. Performance summary
# ========================================================
def summarize(returns):
    equity = (1 + returns).cumprod()
    total_return = equity.iloc[-1] - 1
    sharpe = np.sqrt(252) * returns.mean() / returns.std()
    max_dd = (equity / equity.cummax() - 1).min()
    return total_return, sharpe, max_dd

summary_clean = pd.DataFrame({
    "MLP": summarize(mlp_clean),
    "LSTM": summarize(lstm_clean),
    "CNN-GAF": summarize(cnn_clean)
}, index=["Total Return", "Sharpe Ratio", "Max Drawdown"])

print(summary_clean)
```

```
# ========================================================
# 6. Plot equity curves
# ========================================================
plt.figure(figsize=(14, 6))
plt.plot((1 + mlp_clean).cumprod(), label="MLP (Clean)")
plt.plot((1 + lstm_clean).cumprod(), label="LSTM (Clean)")
plt.plot((1 + cnn_clean).cumprod(), label="CNN-GAF (Clean)")
plt.title("Walk-Forward Backtest with Reduced Leakage")
plt.xlabel("Date")
plt.ylabel("Equity")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer
  super().__init__(**kwargs)
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
                    MLP      LSTM   CNN-GAF
Total Return -0.820641  0.455488 -0.586332
Sharpe Ratio -0.830571  0.518790 -0.300722
Max Drawdown -0.915839 -0.506418 -0.797398
```

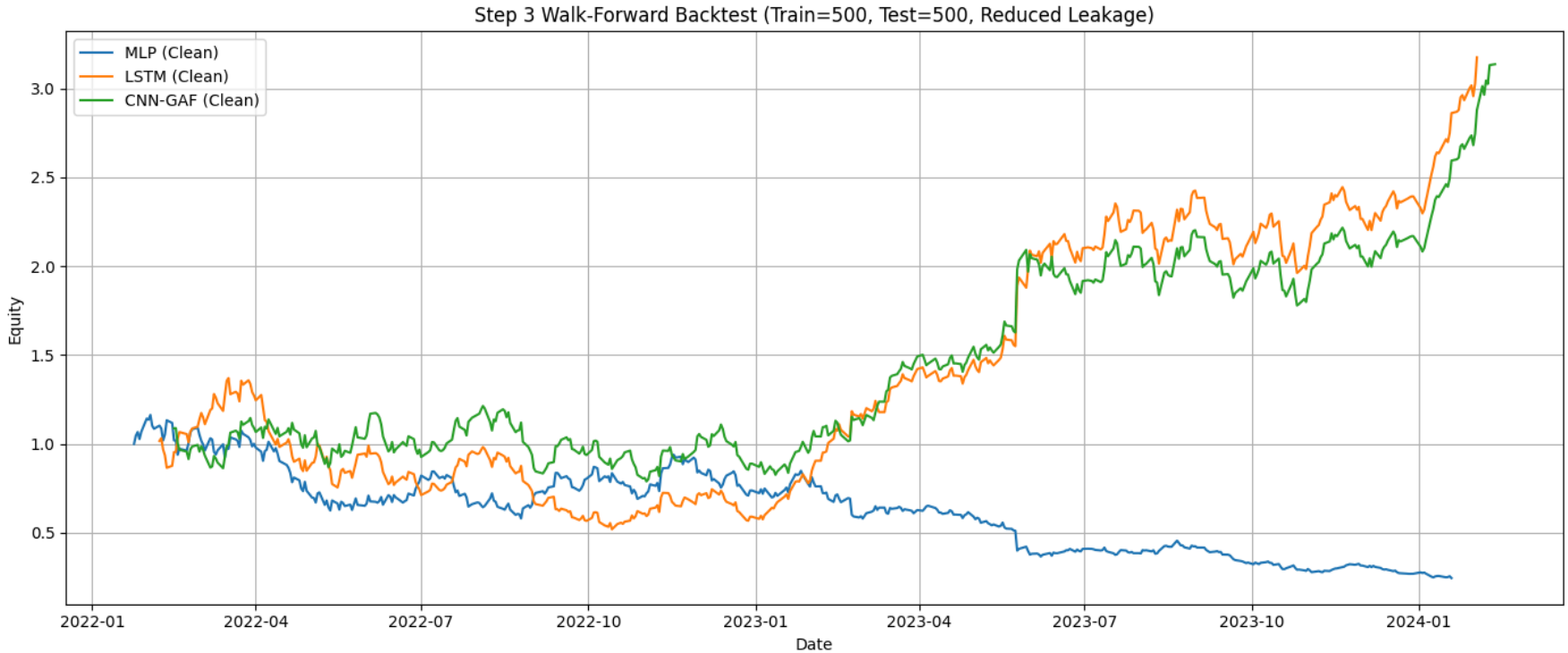


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPRegressor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.optimizers import Adam

# ========================================================
# 0. Clean (causal) label to reduce leakage
# ========================================================
data["clean_label"] = data["log_return"].shift(-1)
data = data.dropna()

# ========================================================
# 1. Feature matrices
# ========================================================
feature_cols = ["ret_lag1", "ret_lag2", "ret_lag3"]
returns = data["log_return"]

# -------- MLP --------
X_mlp = data[feature_cols].values
y_mlp = data["clean_label"].values
returns_mlp = returns

# -------- LSTM --------
seq_len = 10
X_lstm, y_lstm, returns_lstm = [], [], []

for i in range(seq_len, len(X_mlp)):
    X_lstm.append(X_mlp[i - seq_len:i])
    y_lstm.append(y_mlp[i])
    returns_lstm.append(returns.iloc[i])

X_lstm = np.array(X_lstm)
y_lstm = np.array(y_lstm)
returns_lstm = pd.Series(returns_lstm, index=data.index[seq_len:])

# -------- CNN-GAF --------
def gramian_angular_field(ts):
    ts = np.clip(ts, -1, 1)
    phi = np.arccos(ts)
    return np.cos(phi[:, None] + phi[None, :])

scaler = MinMaxScaler(feature_range=(-1, 1))
```

```
X_scaled = scaler.fit_transform(X_mlp)

window_size = 16
X_gaf, y_gaf, returns_gaf = [], [], []

for i in range(window_size, len(X_scaled)):
    channels = []
    for j in range(X_scaled.shape[1]):
        ts_window = X_scaled[i - window_size:i, j]
        channels.append(gramian_angular_field(ts_window))
    X_gaf.append(np.stack(channels, axis=-1))
    y_gaf.append(y_mlp[i])
    returns_gaf.append(returns.iloc[i])

X_gaf = np.array(X_gaf)
y_gaf = np.array(y_gaf)
returns_gaf = pd.Series(returns_gaf, index=data.index[window_size:])

# ========================================================
# 2. Model builders
# ========================================================
def build_mlp():
    return Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(64, 32),
            max_iter=1000,
            random_state=42
        ))
    ])

def build_lstm(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape),
        Dense(1)
    ])
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

def build_cnn(input_shape):
    model = Sequential([
        Conv2D(32, (3, 3), activation="relu", input_shape=input_shape),
        MaxPooling2D((2, 2)),
        Conv2D(64, (3, 3), activation="relu"),
        MaxPooling2D((2, 2)),
        Flatten(),
        Dense(64, activation="relu"),
        Dense(1)
    ])
```

```
    model.compile(optimizer=Adam(0.001), loss="mse")
    return model

# ==========================================================
# 3. Walk-forward backtest with purging and embargo
# ==========================================================
def walk_forward_backtest_purged(
    X, y, returns, model_type,
    train_size=500,
    test_size=500,
    purge_size=10,
    embargo_size=10
):
    strategy_returns = []
    index_out = []
    start = 0

    while start + train_size + purge_size + test_size + embargo_size <= len(X):

        train_start = start
        train_end = start + train_size - purge_size
        test_start = start + train_size + purge_size
        test_end = test_start + test_size

        X_train = X[train_start:train_end]
        y_train = y[train_start:train_end]
        X_test = X[test_start:test_end]
        ret_test = returns.iloc[test_start:test_end]

        if model_type == "mlp":
            model = build_mlp()
            model.fit(X_train, y_train)
            preds = model.predict(X_test)

        elif model_type == "lstm":
            model = build_lstm((X.shape[1], X.shape[2]))
            model.fit(
                X_train, y_train,
                epochs=20,
                batch_size=32,
                shuffle=False,
                verbose=0
            )
            preds = model.predict(X_test).flatten()

        elif model_type == "cnn":
            model = build_cnn(X.shape[1:])
            model.fit(
                X_train, y_train,
```

```
                    epochs=20,
                    batch_size=32,
                    shuffle=False,
                    verbose=0
                )
                preds = model.predict(X_test).flatten()

            signals = np.sign(preds)
            strat_ret = signals * ret_test.values

            strategy_returns.extend(strat_ret)
            index_out.extend(ret_test.index)

            start += test_size

        return pd.Series(strategy_returns, index=index_out)

# ========================================================
# 4. Run clean walk-forward backtests (Train=500, Test=500)
# ========================================================
mlp_clean  = walk_forward_backtest_purged(X_mlp,  y_mlp,  returns_mlp,  "mlp")
lstm_clean = walk_forward_backtest_purged(X_lstm, y_lstm, returns_lstm, "lstm")
cnn_clean  = walk_forward_backtest_purged(X_gaf,  y_gaf,  returns_gaf,  "cnn")

# ========================================================
# 5. Performance summary
# ========================================================
def summarize(returns):
    equity = (1 + returns).cumprod()
    total_return = equity.iloc[-1] - 1
    sharpe = np.sqrt(252) * returns.mean() / returns.std()
    max_dd = (equity / equity.cummax() - 1).min()
    return total_return, sharpe, max_dd

summary_step3 = pd.DataFrame({
    "MLP": summarize(mlp_clean),
    "LSTM": summarize(lstm_clean),
    "CNN-GAF": summarize(cnn_clean)
}, index=["Total Return", "Sharpe Ratio", "Max Drawdown"])

print(summary_step3)

# ========================================================
# 6. Equity curve plot
# ========================================================
plt.figure(figsize=(14,6))
plt.plot((1 + mlp_clean).cumprod(), label="MLP (Clean)")
plt.plot((1 + lstm_clean).cumprod(), label="LSTM (Clean)")
plt.plot((1 + cnn_clean).cumprod(), label="CNN-GAF (Clean)")
```

```
plt.title("Step 3 Walk-Forward Backtest (Train=500, Test=500, Reduced Leakage)")
plt.xlabel("Date")
plt.ylabel("Equity")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer
  super().__init__(**kwargs)
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
                  MLP      LSTM   CNN-GAF
Total Return -0.754951  2.174459  2.136078
Sharpe Ratio -0.992458  1.329780  1.322587
Max Drawdown -0.789318 -0.620810 -0.349662
```